Document Intelligence + Agentic RAG

AI Engineer Intern Assessment · Build Fast with AI

Build a web app that ingests messy, real-world documents, scanned PDFs, handwritten pages, image-heavy reports, tables, long unstructured files extracts content accurately, classifies each document, and powers a chatbot that answers questions with grounded citations showing the exact source page. Security of uploaded documents must be considered and implemented at every layer.

What to Build

---

1. Document Parser

Handle: scanned PDFs, handwritten pages, PDFs with embedded tables, image-heavy reports, plain text. Each page must produce extracted text and a rendered page image store both. Use free Python libraries (e.g. pdfplumber, pdf2image, pytesseract).

2. Document Classifier

After parsing, classify each document using an LLM. Classify across multiple dimensions type, topic, content characteristics, sensitivity level, etc. Output must be structured JSON; decide the schema yourself. Use free/open models or free-tier APIs.

3. Agentic RAG

Build an agent that retrieves relevant chunks and synthesizes an answer. Every answer must include inline citations (document name + page number). If no relevant content exists, say so don't hallucinate. Use any open-source orchestration or RAG library.

4. Chatbot Page

- Text input with multi-turn conversation history
- Each answer shows citations + the cited page rendered as a thumbnail image
- Clicking a thumbnail shows the full page

5. Bulk Upload Page (separate page)

- Upload multiple documents at once to the knowledge base
- Show processing status per file (parsing → classifying → indexed)
- Include 5 + Sample documents in the repo so the chatbot works on first run

6. Voice Input Bonus

Add real-time voice input to the chatbot using any free-tier speech-to-text API. Show live transcript as the user speaks.

Data Security

Uploaded documents may contain sensitive or confidential content. You are expected to think through the threat model yourself and implement security at every layer. Below are the layers to consider how you implement each is your design decision.

Upload layer, Storage layer, Processing layer, API / retrieval layer

In your README, include a short 'Security Decisions' section what you implemented, what you considered but skipped, and what you would add given more time.

Stack

Python backend. Frontend in Next.js (TypeScript). Prefer free and open-source libraries throughout (e.g. PDF parsing, OCR, vector storage, embeddings). Free-tier LLM APIs are fine. No paid services required.

Submission – Deadline (3 Days)

- Public GitHub repository clean structure, README with setup steps, architecture overview, and Security Decisions section
- Deployed working project link (Vercel, Render, or any free host)
- *Optional:* short video walkthrough showing the upload flow, a question with citation, and the source page image

Evaluation

Parsing	OCR works across document types. Tables extracted as structure, not flat text.
Classification	JSON schema is sensible. Documents classified correctly across multiple dimensions.
RAG + Citations	Relevant answers, accurate citations, no hallucination on empty results.
Data Security	Security implemented across upload, storage, retrieval, and API layers. README explains decisions.
UI	Citations + page thumbnails visible. Bulk upload works on a separate page.
Code quality	No secrets in code, clear README, sensible file structure.